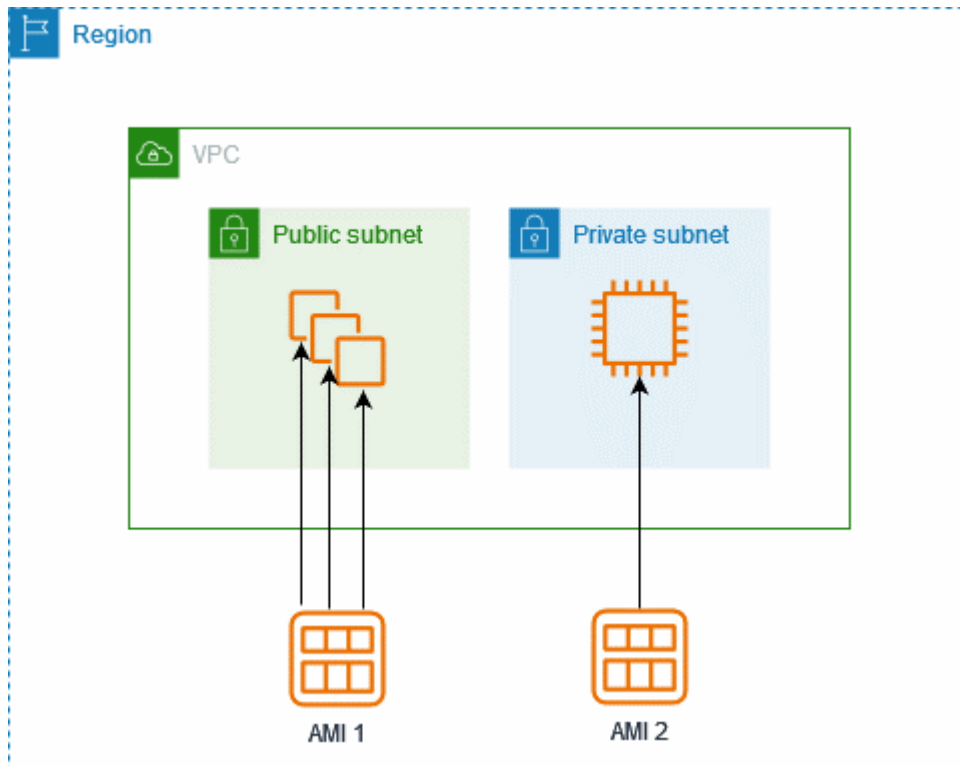


Unit 2

Amazon Machine Images (AMIs) in the deployment of EC2 instances.

Amazon Machine Image (AMI) is a pre-configured template used to launch virtual servers (EC2 instances) in Amazon Web Services. It plays a central role in ensuring that cloud deployments are fast, consistent, secure, and scalable.

AMI-Based Deployment Workflow



Typical Deployment Steps

1. Select an AMI
2. Choose instance type (CPU, RAM, storage)
3. Configure network and security
4. Launch EC2 instance
5. Instance boots using AMI
6. Application becomes available

AMIs help maintain uniform environments across multiple servers.

- ✓ Same OS version

- ✓ Same software stack
- ✓ Same security patches
- ✓ Same configurations

Faster Deployment and Scalability

Rapid Instance Launch

- ❖ Pre-installed software avoids manual setup
- ❖ Instances start in minutes

Auto Scaling Support

Custom AMIs are used in:

- ❖ Auto Scaling Groups
- ❖ Load-balanced systems
- ❖ Cloud-based labs

Create Custom AMI Steps: Organizations and developers often create custom AMIs.

A web application AMI may include:

- 1. Ubuntu OS**
- 2. Apache/Nginx**
- 3. Java/Python**
- 4. Database drivers**
- 5. Project files**

Advantage

1. Benefits
2. One-click deployment
3. Reduced setup time
4. Lower human error

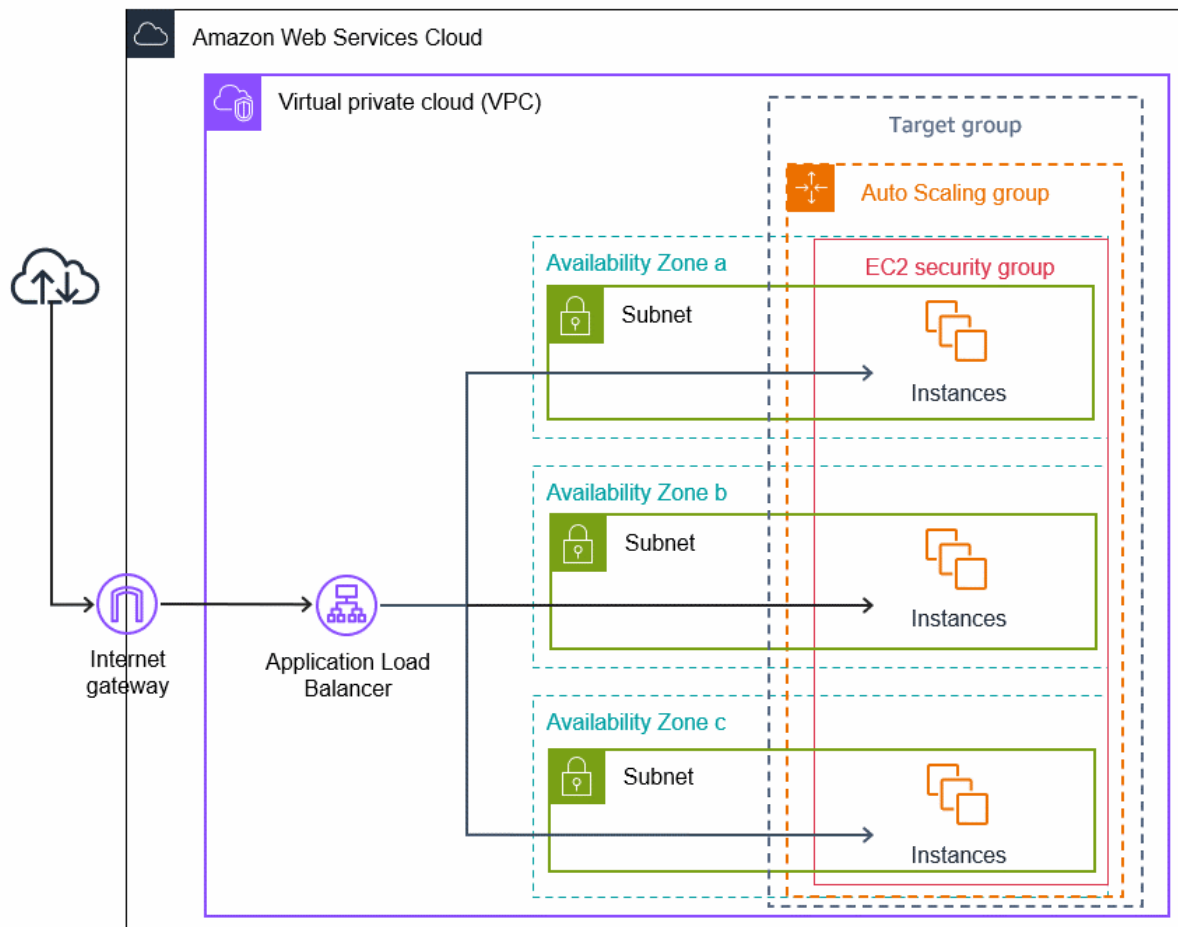
5. Easy replication

Main Usage of AMI

Backup and Disaster Recovery

Cost Optimization

web application architecture using EC2, AMIs, Elastic IPs, Auto Scaling, and Elastic Load Balancing.



Step 1: Design the Architecture

Plan a basic scalable architecture:

Client → Load Balancer → EC2 (Auto Scaling) → Database

It includes the following :

- Multiple EC2 instances
- Auto Scaling Group
- Load Balancer
- AMI for replication
- Elastic IP (if required)

Step 2: Launch a Base EC2 Instance

Open AWS Console → EC2

- Launch Instance
 - Amazon Linux / Ubuntu

- Instance type (e.g., t2.micro)
- Configure:
- Key pair
- Security Group (allow HTTP/HTTPS/SSH)
- Launch the instance

Step 3: Install Web Application on EC2

Connect using SSH and install your application.

Example (Apache Web Server):

```
update -y
install httpd -y
start httpd
```

Step 4: Create an Amazon Machine Image (AMI)

Create an AMI from the configured EC2.

EC2 Dashboard → Instances

Select instance

Actions → Image and templates → Create Image

Provide name → Create

Step 5: Allocate and Attach Elastic IP (Optional)

- Elastic IP gives a static IP address.
- EC2 → Elastic IPs
- Allocate Elastic IP
- Associate with EC2 instance
- Use this when you need:
- Fixed IP for DNS
- Admin server
- Backend access

Step 6: Create Launch Template / Launch Configuration

This defines how new EC2 instances will be created.

1. EC2 → Launch Templates

2. Create Template
3. Select:
 - AMI created
 - Instance type
 - Security group
 - Key pair

Step 7: Create Auto Scaling Group (ASG)

Auto Scaling automatically increases/decreases servers.

1. EC2 → Auto Scaling Groups
2. Create ASG
3. Select Launch Template
4. Choose VPC & Subnets
5. Set capacity:
 - Min: 1
 - Desired: 2
 - Max: 5

Configure Scaling Policy

Example:

- CPU > 70% → Add instance
- CPU < 30% → Remove instance

Step 8: Create Elastic Load Balancer (ELB)

Load Balancer distributes traffic.

1. EC2 → Load Balancers
2. Create Load Balancer
3. Choose:
 - Application Load Balancer (ALB) → For web apps
4. Configure:
 - HTTP (80) / HTTPS (443)
 - Target Group → EC2 instances
5. Attach ASG to Load Balancer